

OpenCV Notes

OpenCV (Open Source Computer Vision Library) is an open-source computer vision and machine learning software library. It provides a common infrastructure for computer vision applications and accelerates the use of machine perception in commercial products. Written in C++ and with bindings for Python, Java, and other languages, OpenCV supports real-time image processing and computer vision tasks.

Key Features

- Cross-platform support (Windows, macOS, Linux, Android, iOS)
 - Real-time image and video processing
 - Integration with deep learning frameworks (TensorFlow, PyTorch, Caffe)
 - GPU acceleration via CUDA and OpenCL
 - Extensive documentation and community support
-

Core Modules of OpenCV

OpenCV is organized into several modules, each focusing on specific functionalities.

1. Core Module (core)

- Basic data structures such as Mat (matrix) for image representation
- Mathematical operations (addition, subtraction, multiplication)
- Random number generation and array manipulation

2. Image Processing Module

- Image filtering (blur, Gaussian blur, median filter)
- Geometric transformations (resize, rotate, warp)
- Edge detection (Canny, Sobel, Laplacian)
- Color space conversions (BGR ↔ Gray, BGR ↔ HSV)
- Histogram computation and equalization

3. High Level GUI Module

- Functions for displaying images and videos
- Keyboard and mouse event handling
- Reading and writing images/videos (imread, imwrite, VideoCapture, VideoWriter)

4. Feature Detection and Description

- Key point detection (SIFT, SURF, ORB, FAST)
- Feature matching using descriptors
- Applications in object recognition and image stitching

5. Video Analysis

- Motion analysis and object tracking
- Background subtraction
- Optical flow estimation (Lucas-Kanade, Farneback)

6. Object Detection

- Pre-trained classifiers for face, eyes, and body detection using Haar cascades
- Integration with deep learning-based detectors (DNN module)

7. Machine Learning

- Implements algorithms like KNN, SVM, Decision Trees, Random Forests, and K-Means
- Useful for classification and clustering tasks

8. Deep Neural Networks (DNN)

- Load and run pre-trained deep learning models
- Supports frameworks like TensorFlow, Caffe, and ONNX
- Enables object detection, segmentation, and classification

Image Representation in OpenCV

Images are represented as multi-dimensional arrays (matrices) using the cv2.Mat or NumPy array in Python.

- **Grayscale image:** 2D array (height × width)
- **Color image (BGR):** 3D array (height × width × 3)
- **Pixel values:** Range from 0 to 255 for 8-bit images

Example:

```
import cv2
img = cv2.imread('image.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imshow('Gray Image', gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Common Image Processing Operations

1. Reading and Writing Images

```
img = cv2.imread('input.jpg')
cv2.imwrite('output.jpg', img)
```

2. Resizing and Cropping

```
resized = cv2.resize(img, (300, 300))
cropped = img[50:200, 100:300]
```

3. Color Conversions

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

4. Blurring and Smoothing

```
blur = cv2.GaussianBlur(img, (5, 5), 0)
```

5. Edge Detection

```
edges = cv2.Canny(img, 100, 200)
```

6. Drawing Shapes and Text

```
cv2.line(img, (0,0), (100,100), (255,0,0), 2)
cv2.putText(img, 'OpenCV', (50,50), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)
```

Object Detection and Tracking

1. Haar Cascade Classifiers

Pre-trained XML files are used for detecting faces, eyes, etc.

```
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
faces = face_cascade.detectMultiScale(gray, 1.3, 5)
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)
```

2. Background Subtraction

Used for motion detection in videos.

```
fgbg = cv2.createBackgroundSubtractorMOG2()
fgmask = fgbg.apply(frame)
```

3. Optical Flow

Tracks motion between frames.

```
flow = cv2.calcOpticalFlowFarneback(prev_gray, next_gray, None, 0.5, 3, 15, 3, 5, 1.2, 0)
```

Integration with Deep Learning

OpenCV's dnn module allows loading and running deep learning models for tasks like object detection and segmentation.

Example (YOLO model):

```
net = cv2.dnn.readNet('yolov3.weights', 'yolov3.cfg')
blob = cv2.dnn.blobFromImage(img, 1/255.0, (416, 416), swapRB=True, crop=False)
net.setInput(blob)
outputs = net.forward(net.getUnconnectedOutLayersNames())
```

Applications of OpenCV

- Face and gesture recognition
 - Object detection and tracking
 - Augmented reality
 - Medical image analysis
 - Autonomous vehicles (lane detection, obstacle avoidance)
 - Industrial inspection and robotics
 - Document scanning and OCR preprocessing
-

Advantages of OpenCV

- Open-source and free to use
 - Large community and extensive documentation
 - Real-time performance
 - Cross-platform compatibility
 - Integration with AI and ML frameworks
-

Limitations

- Limited support for high-level deep learning tasks compared to specialized frameworks
- Requires optimization for large-scale or high-resolution processing
- Some algorithms are CPU-intensive without GPU acceleration